

IN1097 · Tópicos Avançados em Agentes Inteligentes 2

Abordagens de Seleção de Algoritmos (Rankings)

Aula 3 · 20 de agosto de 2026

Profa Juscimara G Avelino

Objetivos da aula

- Entender por que reduzir a informação de performance a um **único "melhor algoritmo"** **descarta informação útil**
- Distinguir as **cinco formas de recomendação** e representá-las em diagramas de Hasse
- Aplicar a **margem de tolerância** para identificar algoritmos com performance comparável
- Justificar por que rankings são adequados quando o orçamento **computacional é desconhecido**

PARTE 1

Por que rankings?

Retomando o Exemplo da aula passada e seus limites

A matriz de performances

Cinco datasets, cinco algoritmos. Cada célula é a acurácia obtida por validação cruzada

DS	SVM_1	SVM_2	DT	RF	KNN
DS1	0,80	0,90	0,75	0,82	0,70
DS2	0,70	0,60	0,80	0,83	0,75
DS3	0,55	0,66	0,65	0,68	0,61
DS4	0,90	0,91	0,92	0,90	0,95
DS5	0,88	0,87	0,80	0,85	0,82

Esta é toda a informação de que dispomos. A pergunta é o que fazer com ela



O passo do meio é o que nos interessa hoje

Reduzindo a um único rótulo

Para treinar um classificador que recomenda um algoritmo, reduzimos cada linha em um só valor:
o **vencedor**

DS	SVM_1	SVM_2	DT	RF	KNN	→	Best
DS1	0,80	0,90	0,75	0,82	0,70		SVM_2
DS2	0,70	0,60	0,80	0,83	0,75		RF
DS3	0,55	0,66	0,65	0,68	0,61		RF
DS4	0,90	0,91	0,92	0,90	0,95		KNN
DS5	0,88	0,87	0,80	0,85	0,82		SVM_1

Cinco números por linha viraram um rótulo. Perdemos informação?

Vamos olhar para DS4

DS4	SVM_1	SVM_2	DT	RF	KNN
Acurácia	0,90	0,91	0,92	0,90	0,95

O KNN venceu. Você diria que os outros são ruins?

Perda 1 — os empates desaparecem

Em DS4, quatro algoritmos ficam entre 0,90 e 0,92. A diferença provavelmente não é significativa

DS4	SVM_1	SVM_2	DT	RF	KNN
Acurácia	0,90	0,91	0,92	0,90	0,95

- O rótulo **Best** registra apenas **KNN**
- A informação de que SVM_1, SVM_2, DT e RF são praticamente **equivalentes** entre si é **descartada**
- Se o meta-modelo errar e recomendar DT em vez de KNN, o erro custa 0,03 — mas o meta-modelo é penalizado como se tivesse errado tudo

Perda 2 — ruído no meta-modelo

- Quando vários algoritmos são **estatisticamente indistinguíveis**, escolher um deles como "o melhor" é uma decisão arbitrária
- Em outro dataset muito parecido, o ruído da validação cruzada pode fazer outro algoritmo vencer
- O meta-modelo então vê dois **datasets quase idênticos com rótulos diferentes**
- **Resultado: ele tenta aprender uma fronteira que não existe nos dados**

O rótulo arbitrário vira ruído sistemático no meta-nível

Perda 3 — os bons concorrentes somem

- O usuário não vai testar apenas um algoritmo. Ele tem tempo para testar três, cinco, talvez dez
- Um sistema que devolve apenas o primeiro colocado não diz nada sobre o que testar em seguida
- E se o primeiro colocado falhar no dataset real do usuário, ele fica sem orientação nenhuma
- **A informação sobre o segundo, terceiro e quarto lugares existia — e foi descartada**

Há outras formas de definir o alvo

PARTE 2

As cinco formas de recomendação

O que um sistema pode entregar

Supondo o portfólio $\{a_1, a_2, \dots, a_6\}$, o sistema pode recomendar ao usuário:

1 Melhor algoritmo de um conjunto

Um único item

2 Subconjunto dos top algoritmos

Poucos itens, sem ordem

3 Ranking linear

Todos os itens, ordem estrita

4 Ranking quasi-linear (fraco)

Todos os itens, com empates

5 Ranking incompleto

Só parte dos itens

As cinco formas

Table 2.1: Examples of different forms of recommendation

1. Best in a set	a_3					
2. Subset	$\{a_3, a_1, a_5\}$					
3. Linear and complete ranking	Rank					
	1	2	3	4	5	6
	a_3	a_1	a_5	a_6	a_4	a_2
4. Quasi-linear and complete ranking	$\overline{a_3}$	$\overline{a_1}$	a_5	$\overline{a_6}$	$\overline{a_4}$	a_2
5. Linear and incomplete ranking	a_3	a_1	a_5	a_6		

Tabela 2.1 de Brazdil et al (2022)

Uma notação visual: diagramas de Hasse

- Cada nó é um algoritmo
- Cada aresta dirigida representa a relação "significativamente melhor que"
- Uma corrente vertical simples é um ranking linear: cada algoritmo é melhor que o seguinte
- Ramificações indicam que não há evidência para ordenar dois algoritmos entre si
- **A forma do diagrama revela, de relance, quanta informação o ranking realmente carrega**

Os quatro casos em diagramas de Hasse

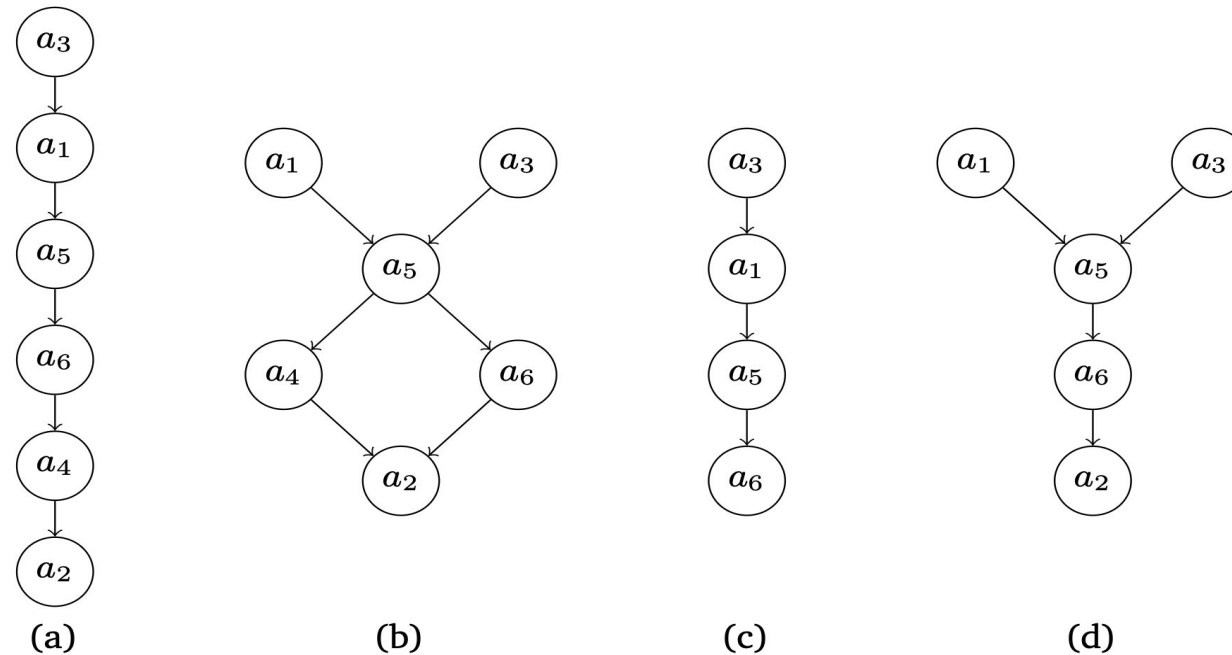


Fig. 2.1: Representation of rankings using Hasse diagrams: (a) complete linear ranking; (b) complete quasi-linear ranking; (c) incomplete linear ranking; (d) incomplete quasi-linear ranking

FORMA 1

Melhor algoritmo de um conjunto

Formalmente, um ranking de tamanho 1

Forma 1 — a ideia

- Consiste em identificar o algoritmo que se espera obter a melhor performance no conjunto de algoritmos de nível base
- Note que isto é formalmente um ranking de tamanho 1
- Intuitivamente, é o que a maioria dos cientistas de dados faz: aplicar seu algoritmo favorito primeiro em um novo dataset

Forma 1 — como construir sem aprendizado

- Identifica-se o melhor algoritmo em cada dataset conhecido
- A informação reunida precisa ser agregada
- Uma possibilidade: usar o algoritmo que foi o melhor no maior número de datasets

Dataset	DS1	DS2	DS3	DS4	DS5
Melhor	SVM_2	RF	RF	KNN	SVM_1

RF vence em dois dos cinco. Sem aprender nada, já temos um recomendador

Este é o baseline contra o qual qualquer sistema de meta-aprendizagem precisa se comparar

Forma 1 — a limitação

- Como esta estratégia não usa busca, há sempre a chance de o algoritmo recomendado não ser de fato o melhor no caso concreto
- Consequentemente, podemos obter um resultado subótimo
- O usuário testa um algoritmo, obtém um número, e não tem base de comparação

Se você tivesse tempo para testar três algoritmos em vez de um, o que gostaria de receber do sistema?

FORMA 2

Subconjunto dos top algoritmos

Um conjunto pequeno, sem ordem interna

Forma 2 — a ideia

- Recomendam um subconjunto — normalmente pequeno — de algoritmos para os quais se espera boa performance no dado problema
- A ideia é fornecer um subconjunto bem menor que o conjunto original, no qual haja probabilidade alta de o melhor algoritmo estar de fato incluído
- O usuário pode então aplicar uma estratégia de busca dentro desse subconjunto
- Um sistema simples, sem aprendizado, pode recomendar os top algoritmos considerando o ranking médio

Forma 2 — a estratégia ingênua

- Identificar o melhor algoritmo em cada dataset
- Se houver empate, escolher simplesmente o primeiro na ordem em que aparecem
- Agregar tomando a união de todos os algoritmos identificados
- **Com k datasets de treino, chegamos a um subconjunto de no máximo k elementos**

Melhores por dataset: SVM_2 · RF · RF · KNN · SVM_1

União: { SVM_1, SVM_2, RF, KNN }

Forma 2 — o defeito da estratégia ingênua

Este método ignora todos os algoritmos que alcançam performance comparável à do melhor

Consequentemente, há a chance de o subconjunto selecionado não incluir o verdadeiro melhor

- Voltemos a DS4: apenas KNN entra na união, embora DT estivesse a 0,03 de distância
- A noção de "ter boa performance" em um dataset é tipicamente definida em termos relativos
- Um modelo que acerta 50% dos casos é excelente em alguns datasets e medíocre em outros
- **Precisamos de um critério para decidir quem mais é "suficientemente bom"**

Como achar performances comparáveis

Assumindo que o melhor algoritmo a^* já foi identificado, existem outros com performance comparável?

Via A — teste estatístico

- Testa-se se a performance de cada candidato é significativamente pior que a do melhor
- Entra no subconjunto "bom" quem não for significativamente pior
- Testes paramétricos (t-test) ou não-paramétricos (Wilcoxon signed-rank)
- Exige validação cruzada: precisa da informação obtida em cada fold

Via B — margem de tolerância

- Alternativa quando a informação por fold não está disponível
- Define-se uma margem relativa à performance do melhor algoritmo naquele dataset
- Todos dentro da margem são considerados bons também
- Pressupõe que os erros são normalmente distribuídos

A margem de tolerância

$$\left(e_{min}, e_{min} + k \sqrt{\frac{e_{min} (1 - e_{min})}{n}} \right)$$

Eq 2.1 *Brazdil et al (2022)*

e_min

Erro do melhor algoritmo
naquele dataset

n

Número de exemplos do
dataset

k

Nível de confiança definido
pelo usuário — controla o
tamanho da margem

A margem na prática

Dataset com $n = 500$ exemplos. O melhor algoritmo erra 12% dos casos, logo $e_{\min} = 0,12$

k	Margem superior	Erro tolerado	Interpretação
1	0,1345	até 13,45%	Critério estrito — poucos algoritmos entram
2	0,1491	até 14,91%	Critério intermediário
3	0,1636	até 16,36%	Critério frouxo — o subconjunto infla

Quem escolhe k escolhe o tamanho da recomendação

Não é um detalhe técnico: é uma decisão sobre quanto risco o usuário aceita correr

Agregando os subconjuntos

- Os subconjuntos gerados em cada dataset precisam ser agregados — por exemplo, tomando a união
- Os algoritmos do subconjunto final podem ser ordenados por em quantos datasets aparecem
- Se a_i aparece em vários subconjuntos e a_j em apenas um, a_i recebe rank mais alto: a probabilidade de a_i ter boa performance no dataset alvo é maior
- **Vantagem: a fase de busca envolve mais de um algoritmo, aumentando a chance de o verdadeiro melhor estar incluído**

FORMAS 3, 4 E 5

Rankings

Ordem completa, com empates, e incompleta

Forma 3 — ranking linear

- O sistema recomenda um ranking de todos os algoritmos
- Tipicamente, a ordem indicada é a ordem a ser seguida na fase de experimentação
- É chamado linear porque os ranks são diferentes para todos os algoritmos
- É completo porque todos os algoritmos têm seu rank definido

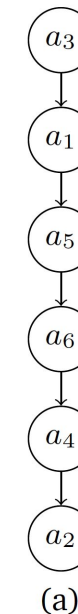


Fig 2.1(a) — ranking linear completo

Forma 3 — a limitação

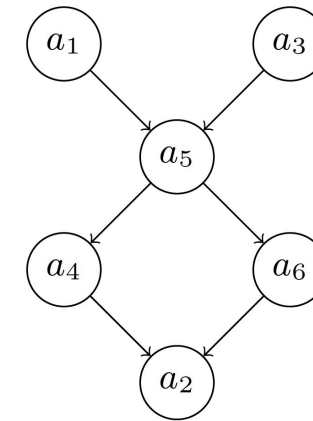
Um ranking linear não consegue representar o caso em que dois algoritmos empatam num dataset

Isto é, quando a performance dos dois não é significativamente diferente

- O ranking é obrigado a colocar um antes do outro, mesmo sem evidência para isso
- É exatamente o mesmo problema da Forma 1, agora distribuído por todas as posições
- **A Forma 4 existe para resolver isso**

Forma 4 — ranking quasi-linear

- Sempre que dois ou mais algoritmos empatam, o ranking quasi-linear (ou fraco) pode ser utilizado
- Representa situações em que os dados não permitem distinguir a performance relativa — ou em que os algoritmos são de fato indistinguíveis
- O problema é resolvido assinalando o mesmo rank a todos os algoritmos empatados
- Na Tabela 2.1, a linha acima dos nomes (como em a_3 a_1) indica que a diferença não é significativa



(b)

Fig 2.1(b) — ranking quasi-linear completo

Forma 4 — um sistema que faz isso

- Brazdil, Soares e Pereira (2001) propuseram um método de meta-aprendizagem que recomenda rankings nessa forma
- É uma adaptação da abordagem kNN de ranking
- O método identifica algoritmos com performance equivalente e inclui apenas um deles na recomendação
- **Efeito prático: a lista entregue ao usuário fica mais curta, sem perder cobertura**

Forma 5 — ranking incompleto

Tanto o ranking linear como o quasi-linear podem ser incompletos. Duas situações bem diferentes:

Situação 1 — exclusão deliberada

- Alguns algoritmos não foram avaliados por motivos diversos: erros de execução, custo computacional elevado, dificuldades técnicas
- Neste caso, deve-se simplesmente tratar o ranking incompleto como se fosse completo

Situação 2 — portfólio em crescimento

- Novos algoritmos são desenvolvidos e precisam ser adicionados ao conjunto existente
- A execução nos datasets pode ser gradual — os metadados não precisam estar completos
- Surge então a questão de quais testes conduzir primeiro

As cinco formas

Forma	Quantos itens	Tem ordem?	Representa empate?	Custo p/ o usuário
1 Melhor de um conjunto	1	—	Não	Mínimo
2 Subconjunto dos top	Poucos	Não	Implicitamente	Baixo
3 Ranking linear	Todos	Estrita	Não	Variável
4 Ranking quasi-linear	Todos	Com empates	Sim	Variável
5 Ranking incompleto	Parte	Sim	Depende	Variável

Quanto mais informação a forma carrega, mais o usuário pode decidir por conta própria

Por que rankings, afinal?

O sistema de meta-aprendizagem pode ser desenvolvido sem qualquer informação sobre quantos algoritmos base o usuário testará

Este número depende dos recursos computacionais disponíveis e da importância de obter boa performance

Se o tempo é o fator crítico

Somente poucas alternativas devem ser selecionadas

Se a acurácia é o fator crítico

Mais algoritmos devem ser examinados, o que aumenta a chance do melhor resultado

A intuição central

Se o objetivo é identificar o algoritmo com a melhor performance, é mais importante saber as performances relativas do que as absolutas

- Predizer que um algoritmo alcançará 0,847 de acurácia é difícil e, no fim, irrelevante
- Predizer que ele ficará à frente dos outros quatro é mais fácil e é o que o usuário precisa
- **A tarefa de recomendação vira a tarefa de ordenar algoritmos segundo sua performance predita**

Desenhando rankings

Acurácias de 6 algoritmos em 4 datasets. Todos os datasets têm $n = 500$ exemplos.

- 1 Para d_1 , construa o ranking linear completo e desenhe o diagrama de Hasse.
- 2 Calcule a margem de tolerância para d_1 com $k = 2$, sabendo que $e_{\min} = 1 - 0,88$.
- 3 Redesenhe: quem cai dentro da margem empata. Construa o ranking quasi-linear.
- 4 Refaça o passo 2 com $k = 1$. O conjunto de empatados muda? Por quê?
- 5 Suponha que a_4 e a_6 não puderam ser executados. Desenhe o ranking incompleto.

	d_1	d_2	d_3	d_4
a_1	0,88	0,71	0,64	0,90
a_2	0,87	0,83	0,59	0,76
a_3	0,86	0,79	0,81	0,72
a_4	0,79	0,85	0,77	0,88
a_5	0,72	0,68	0,73	0,81
a_6	0,65	0,74	0,68	0,69

Compare os três diagramas: qual deles você entregaria ao usuário, e o que cada um esconde?

- Reduzir tudo a "Best" descarta três coisas: empates, robustez do meta-modelo e informação sobre concorrentes
- Existem cinco formas de recomendação, que diferem em quanta informação transmitem
- Diagramas de Hasse tornam visível a estrutura de um ranking
- Decidir quem tem "performance comparável" exige um critério — teste estatístico ou margem de tolerância
- Rankings servem bem porque não exigem saber, de antemão, o orçamento do usuário